

The learning problem, and a line learned by descent

Unit 1 gave you a way to walk downhill. Today we decide what the hill is. Six points, one straight line through them — first solved exactly with a formula, then thrown away and found again by descending, because only the second method survives contact with a neural network.

Unit 2 · From scratch

80 minutes

Lab 7 · exact fit, then descent

No libraries beyond NumPy

Five words

The exact fit

The loss surface

By descent

Lab 7

Before Tuesday

PART 1 · 15 MINUTES

Five words that define every learning problem

Feature, label, model, parameter, loss. Everything this term is built out of these.

1

On Tuesday you minimised $f(x) = x^2$. Nobody hands you that function in practice. What you have instead is data — examples of inputs and the outputs that go with them — and a family of functions you are willing to try. **Learning is the act of choosing, from that family, the member that fits the data best.** Five words make that sentence precise.

Word	Meaning	In today's example
Feature x	what you know about an example	hours of revision a student did (0 to 5)
Label y	what you want to predict	the mark they received, out of 12
Model	a family of functions, with dials	$\hat{y} = wx + b$ — every straight line
Parameters	the dials the data will set	w (slope) and b (intercept)
Loss	one number saying how wrong a choice of dials is	mean squared error over the six points

THE SENTENCE TO CARRY THROUGH UNIT 2

A model has parameters. A loss scores any setting of the parameters against the data.

Training is gradient descent on the loss — the same walk you did on Tuesday, except the ground is now made of data.

Training data and test data

A model that fits the data it was trained on has proved very little: a lookup table does that perfectly. What we care about is the *next* student, the one whose mark we have not seen. So before any fitting, we set some examples aside and do not touch them until the end. The examples used to choose the parameters are the **training set**; the ones held back to measure honestly are the **test set**.



This is a promise, not a formality

The moment test data influences a single decision — a learning rate, a feature, a stopping point — it stops being a test. Session 20 shows what that does to a score, by committing the mistake on purpose. Today, with six points, we keep all six for training; the idea matters more than the split.

Six points, fitted exactly

Least squares has a closed form. We use it once, so you have seen a fitted model before you see an iterative one.

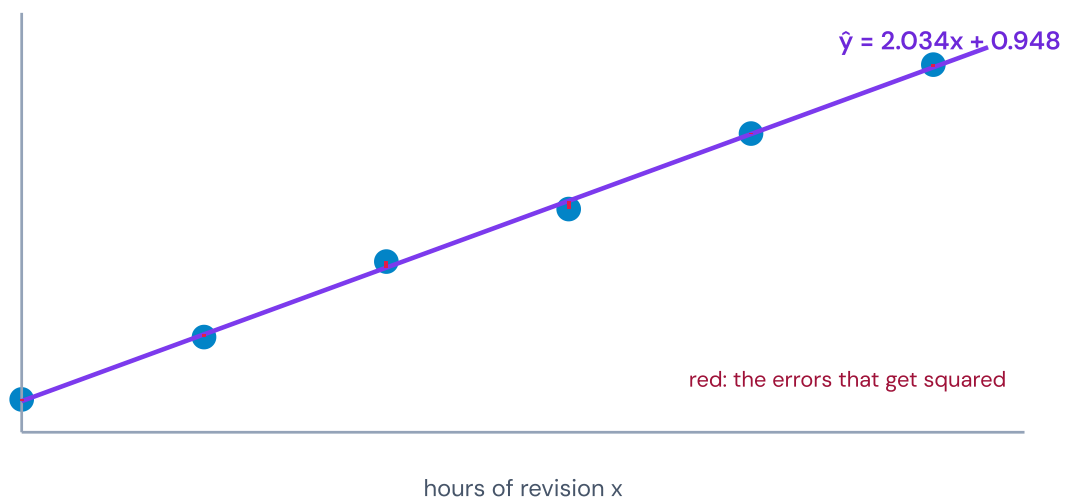
2

the six points – the same six as Lab 7

PYTHON

```
import numpy as np
x = np.array([0., 1., 2., 3., 4., 5.])      # hours of revision
y = np.array([1.0, 2.9, 5.2, 6.8, 9.1, 11.2]) # mark out of 12
```

LEAST SQUARES CHOOSES THE LINE THAT MAKES THE RED GAPS, SQUARED AND AVERAGED, AS SMALL AS POSSIBLE



MEAN SQUARED ERROR – THE LOSS FOR TODAY

$$L(w, b) = \text{mean}((w \cdot x + b - y)^2)$$

Square each gap so that errors above and below the line cannot cancel, and so that a large miss costs much more than a small one. Average them so the number does not grow just because there are more points.

For a straight line, calculus gives the minimum in closed form. With $\bar{x}$ and $\bar{y}$ the means, the slope is the covariance of x and y divided by the variance of x , and the line passes through the point of means:

the exact answer

PYTHON

```
xm, ym = x.mean(), y.mean()
w = np.sum((x - xm) * (y - ym)) / np.sum((x - xm) ** 2)
b = ym - w * xm
print(round(w, 4), round(b, 4), round(np.mean((w * x + b - y) ** 2), 4))
```

output

OUTPUT

```
2.0343 0.9476 0.0188
```

Read the last number carefully. The best possible line still has a loss of **0.0188**, not zero. Real measurements are noisy; no line passes through all six. *A loss that cannot reach zero is normal, and chasing zero is how models learn noise* — that sentence returns as the whole of session 22.

PART 3 · 12 MINUTES

The loss is a surface, and we are standing on it

Two parameters means the hill from Tuesday has become a bowl.

3

Pick any w and b and you get a number: the loss. At $w = 0, b = 0$ — the flat line through the origin — the loss is **48.49**. At $w = 2, b = 1$, a good guess by eye, it is **0.0233**. At the exact answer it is **0.0188**. Every pair of dials is a point on a surface; the exact answer is the bottom of the bowl.

w	b	Loss	What it looks like
0	0	48.49	flat line along the axis — terrible
2	1	0.0233	a sensible guess by eye
2.0343	0.9476	0.0188	the bottom of the bowl

The gradient has two components now, one per dial. Each is Tuesday's chain rule applied to the squared error:

THE TWO SLOPES OF THE MEAN SQUARED ERROR

$$\partial L / \partial w = 2 \cdot \text{mean}(e \cdot x) \quad \partial L / \partial b = 2 \cdot \text{mean}(e)$$

Compute the error once, $e = w \cdot x + b - y$, then both slopes from it. A step moves w and b together — never one and then the other.

Before we descend

Answer both without running anything.

Q1. At $w = 0$, $b = 0$, is $\partial L / \partial w$ positive or negative? Every error $e = 0 - y$ is negative, and every x is non-negative, so $\text{mean}(e \cdot x)$ is negative. The slope is negative, so the update $w \leftarrow w - \eta \cdot (\text{negative})$ increases w — towards 2. The sign points away from the minimum, exactly as on Tuesday.

Q2. Why does the w gradient carry an x but the b gradient does not? Because b is added to every prediction with coefficient 1, while w is multiplied by x . The chain rule brings that coefficient out. Points with large x therefore push harder on the slope — which is why, in Lab 7's extension, a large learning rate misbehaves sooner than Tuesday's cliff at 1.0.

PART 4 · 15 MINUTES

Throw the formula away, and descend

The same six points, the same bowl, reached by walking.

4

Tuesday's update, twice per step

PYTHON

```
w, b, lr = 0.0, 0.0, 0.02
for epoch in range(1000):
    e = w * x + b - y
    w, b = w - lr * 2 * np.mean(e * x), b - lr * 2 * np.mean(e)
print(round(w, 4), round(b, 4), round(np.mean((w * x + b - y) ** 2), 4))
```

output

OUTPUT

2.0343 0.9476 0.0188

Identical to four decimals. Descent reached the bottom of the bowl without ever being told where it was — it only ever looked at the slope under its feet. It took a while: the loss was within 1% of the exact minimum after **224** of those thousand steps.

WHY KEEP THE SLOW METHOD?

The formula exists because a straight line with squared error is a very special problem. On Tuesday next week, the prediction goes through a sigmoid, and there is no formula any more. By session 13 the model has thirteen parameters and by session 16 over a thousand. **Descent does not care.** It needs a loss and a slope, and nothing else — which is why every network you will ever train is trained this way.

LAB 7 · 20 MINUTES IN CLASS

The exact fit and the descended fit

5

Lab 7 is a separate handout — `CSC4315_Lab07_Exact_Fit_And_Descended_Fit`. You write the closed form, then the two gradient lines, then find how many epochs it took to get within 1% of the exact answer. Data, plotting and the checks are given; the two ideas are yours.



The rules have not changed

Finish in the room. Hand up after three minutes stuck. And watch for the one bug everyone meets: updating `w`, then recomputing the error before updating `b`. It runs, and it is a different algorithm.

Before Tuesday

1. **Finish Lab 7.**
2. **Be able to state, without notes:** the five words, what the test set is for, and the two MSE gradients.
3. **Think about this:** if the label were *pass* or *fail* instead of a mark, what would go wrong with a straight line?



Session 11 — Tuesday 13 October, 16:00

From predicting a number to predicting a class. The activation function appears for the first time, with a reason attached; so does cross-entropy, and the reason squared error is the wrong loss for a decision.